Лабораторная работа: защита хостинговой системы от сетевых атак

Среда выполнения: виртуальная машина VirtualBox / VMware / QEMU, Debian 13 (Trixie). Допускается любой Linux-дистрибутив (Debian, Ubuntu, Astra и т. п.) — учебная ВМ не обязательна; команды могут незначительно отличаться в зависимости от дистрибутива. **Формат:** практическая работа по типу Nebula / Lin.Security — «атака → защита → проверка»

Что нужно для зачёта

№	Задание	Статус
1	SSH: подбор пароля	**обязательно**
2	Fail2ban как компенсирующий контроль	**обязательно**
3	Выход за пределы Docker-контейнера	**на выбор** — достаточно одного из заданий 3 и 4
4	Избыточно открытый сервис и nftables	**на выбор** — достаточно одного из заданий 3 и 4

По каждому выбранному заданию нужны эксплуатация, защита и проверка — со скриншотами и видимой меткой `[LAB:...]`.

Задания **независимы по подготовке**: стенд для каждого задания ставится отдельной командой (см. п. 0.3), и скрипт устанавливает только то, что нужно этому заданию. Если вы делаете только задание 4 (firewall), Docker на машине не появится вообще. Задание 2 (fail2ban) выполняется на том же SSH-стенде, что и задание 1.

Виртуальная машина

- **Готовый образ ВМ (рекомендуется):** скачать
- **ОС:** Debian 13 (Trixie), минимальная установка, без десктопа
- **Логин:** `root`
- **Пароль:** `qwe123!@#`
- **Сетевой адаптер:** Bridged или Host-only (важно для заданий 1, 2 и 4 — см. п. 0.5)

0. Общие сведения

0.1. Подготовка виртуальной машины

Стенд можно подготовить двумя способами: взять готовый образ (быстрее) или установить Debian 13 с нуля (даёт полный контроль над составом системы).

Вариант А. Готовая виртуальная машина (рекомендуется)

1. Скачайте образ Debian 13 по адресу **https://disk.yandex.ru/d/w00qwC9gyg79OQ**.
2. Импортируйте образ в свой гипервизор (в VirtualBox — «Файл → Импорт конфигурации»; в VMware/QEMU откройте прилагаемый файл виртуальной машины).
3. Включите ВМ и войдите в систему: логин root, пароль qwe123!@#.
4. Проверьте, что сетевой адаптер работает в режиме **Bridged** или **Host-only**, а не NAT (см. п. 0.5).

Вариант В. Установка Debian 13 с нуля (альтернатива)

1. Скачайте установочный образ Debian 13 (Trixie) с https://www.debian.org/download и создайте новую ВМ в VirtualBox / VMware / QEMU.
2. Установите систему в минимальной конфигурации, **без десктопа**.
3. Настройте сетевой адаптер в режиме **Bridged** или **Host-only**, а не NAT.
4. Войдите в систему и переходите к следующему шагу.

0.2. Скачивание скриптов лабораторной

Скрипт подготовки стенда и файлы уязвимых приложений лежат в репозитории курса, поэтому копировать их вручную не нужно — скачайте их прямо на ВМ:

```
# Создаём каталоги для скрипта и приложений
mkdir -p ~/lab_network_protection/scripts ~/lab_network_protection/app
cd ~/lab_network_protection

# Адрес репозитория курса (ветка 2026_fall) и каталога лабораторной
REPO=https://raw.githubusercontent.com/james116blue/os-protection/2026_fall
BASE=$REPO/lab_network_protection

# Скачиваем скрипт подготовки стенда и два приложения
wget -O scripts/lab_network_protection_setup.sh \
     "$BASE/scripts/lab_network_protection_setup.sh"
wget -O app/vulnerable_app.py       "$BASE/app/vulnerable_app.py"
wget -O app/debug_status_service.py "$BASE/app/debug_status_service.py"
```

```
# Проверяем, что все три файла на месте
ls -l scripts app

# Делаем скрипт исполняемым
chmod +x scripts/lab_network_protection_setup.sh
```

> Если wget не установлен: `apt-get update && apt-get install -y wget` (или замените `wget -O` на `curl -fsSL -o`).

0.3. Подготовка стенда под конкретное задание

Скрипт подготовки принимает вторым аргументом **задание**, под которое нужно поднять стенд:

```
# Общий вид команды
sudo ./scripts/lab_network_protection_setup.sh "<ваш идентификатор>" <задание>

# Примеры
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" ssh       # задание
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" fail2ban  # задание
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" docker    # задание
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" firewall  # задание
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" all       # всё сраз
```

Что именно ставит каждый вариант:

Задание	Что разворачивает скрипт	Что НЕ ставит
ssh (1)	пользователь hacker, слабый sshd_config (PermitRootLogin yes, PasswordAuthentication yes, MaxAuthTries 100), пароль root:toor123, flag1, словарь ~/wordlist.txt, nmap/hydra/nc	Docker, debug-службу на 9000

Задание	Что разворачивает скрипт	Что НЕ ставит
`fail2ban` (2)	снова включает вход по паролю и вход root в `sshd` (сценарий «пароль отключить нельзя»), проверяет `root:toor123`, словарь и hydra	Docker, debug-службу на 9000; сам fail2ban студент ставит вручную в шаге 2
`docker` (3)	`docker.io`, `/opt/vulnerable_app`, образ `lab-vuln-app` (с `ping`), контейнер `vuln_app` с `--privileged` и `-v /:/hostfs`, `flag3`	debug-службу на 9000
`firewall` (4)	`lab-debug.service` на `0.0.0.0:9000`, `flag2`, `nmap`/`nc`	**Docker вообще не устанавливается**
`all`	всё перечисленное	—

Базовые вещи (метка `LAB_TOKEN`, каталог флагов, пользователь `hacker`, словарь, сетевые утилиты) ставятся всегда — они нужны любому заданию. Повторный запуск скрипта с другим заданием безопасен: он доустановит недостающее и не тронет уже настроенное.

Скрипт запускается **один раз на задание** самим студентом — его не нужно разбирать команду за командой, в отличие от команд внутри самих заданий, где важно понимать каждый шаг.

`sudo` запросит пароль вашей учётной записи. Если вы уже работаете под `root`, `sudo` можно не писать.

Скрипт выведет уникальную **метку лабораторной работы** (`LAB_TOKEN`) — 16-символьный hex-идентификатор на основе вашего идентификатора, имени хоста, времени запуска и случайного числа. Метка автоматически встраивается в приглашение командной строки (`PS1`) всех пользователей и в `/etc/motd`.

0.4. Правило анти-плагиата

На каждом скриншоте в отчёте должна быть видна метка `[LAB:xxxxxxxxxxxxxxxx]` в начале строки приглашения терминала. Скриншоты без этой метки, с чужой меткой или с меткой, не совпадающей с той, что выдал ваш собственный запуск скрипта, не засчитываются.

Метку также можно вывести командой:

```
echo "$LAB_TOKEN"
cat /etc/lab-token
```

Откройте новый терминал (или выполните `exec bash -l`) после запуска скрипта, чтобы метка появилась в приглашении.

0.5. Откуда вести атаку и как определить свой IP-адрес

В заданиях 1, 2 и 4 атака выполняется по сети, и трафик **на loopback (`127.0.0.1`) ведёт себя иначе**, чем трафик на внешний интерфейс:

- Правила nftables для входящего трафика на внешнем интерфейсе не применяются к трафику на loopback.
- fail2ban по умолчанию игнорирует адреса из `127.0.0.1/8`.

Поэтому атаку нужно вести **на собственный IP-адрес ВМ из общей сети**, а не на `127.0.0.1`. Чтобы понимать, что именно вы вводите (а не просто скопировать результат скрипта), определите адрес вручную:

```
ip a
```

В выводе найдите активный сетевой интерфейс (обычно `eth0`, `enp0s3` или похожий — не `lo`) и строку вида:

```
2: eth0: ... state UP ...
    inet 192.168.56.10/24 ... scope global eth0
```

Число перед `/24` — это ваш IP-адрес. Запишите его и явно укажите в переменной (замените `192.168.56.10` на то, что увидели у себя):

```
TARGET_IP=192.168.56.10
```

Эту переменную дальше используют все команды в заданиях. Если вы откроете новый терминал, переменную нужно будет задать снова — это обычное поведение shell-переменных, они не сохраняются между сессиями.

Атаку можно вести как с самой ВМ (на собственный внешний IP), так и с хост-машины студента, если сетевой адаптер ВМ

настроен в режиме Bridged или Host-only (не NAT). В задании 3 (Docker) этот момент не актуален — там можно использовать и `127.0.0.1:8080`, работает в обоих случаях, потому что там мы атакуем приложение, а не проверяем фильтрацию на уровне сетевого интерфейса.

0.6. Роли

- **«Нарушитель»** — учётная запись `hacker` (пароль `hacker`), созданная скриптом.
- **«Администратор»** — ваша обычная учётная запись с правами sudo (или root).

Переключение — `su - hacker` / выход обратно, либо два отдельных терминальных сеанса.

0.7. Структура каждого задания

1. **Подготовка** — какой вариант скрипта запустить (ставит только нужное этому заданию).
2. **Теория** — что это за механизм защиты, зачем он нужен и как устроен внутри.
3. **Эксплуатация** — воспроизвести атаку, получить флаг/результат.
4. **Защита** — внести исправление от имени администратора, с объяснением каждой настройки.
5. **Проверка** — повторить атаку и зафиксировать, что она больше не работает.

0.8. Полезные команды аудита

```
# Что сейчас слушает сеть и на каком интерфейсе
ss -tlnp

# Текущие правила nftables
nft list ruleset

# Конфигурация sshd целиком, как её видит сам sshd (после всех include)
sshd -T

# Запущенные контейнеры и их конфигурация
docker ps
docker inspect <container> | less

# Статус fail2ban по конкретному jail
fail2ban-client status sshd
```

Задание 1 (обязательно). SSH: подбор пароля

Подготовка стенда

```
cd ~/lab_network_protection
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" ssh
```

Скрипт создаёт пользователя hacker, приводит sshd в намеренно уязвимое состояние (PermitRootLogin yes, PasswordAuthentication yes, MaxAuthTries 100), задаёт root:toor123, кладёт словарь ~/wordlist.txt и ставит hydra, nmap, nc. Docker и debug-служба на 9000 при этом не устанавливаются.

Теория

Аутентификация по паролю и по ключу

SSH поддерживает несколько способов аутентификации, два основных — пароль и публичный ключ:

- **По паролю**: клиент отправляет серверу пароль (через зашифрованный канал, но сам пароль — это просто секретная строка), сервер сравнивает его с хранимым хешем. Пароль можно подобрать — переслать много вариантов, пока один не совпадёт. Скорость подбора ограничена только тем, как быстро сервер отвечает на попытки.
- **По ключу (публичный/приватный)**: на клиенте генерируется пара ключей — приватный (остаётся только у клиента, никогда не передаётся) и публичный (кладётся на сервер, в ~/.ssh/authorized_keys). При подключении сервер присылает клиенту случайное число, клиент подписывает его приватным ключом, сервер проверяет подпись публичным ключом. Секрет (приватный ключ) никогда не передаётся по сети и не может быть подобран перебором за разумное время (ключи ed25519/RSA-4096 — это числа такого размера, что перебор физически не успеет завершиться).

Отсюда ключевой вывод для защиты: отключение парольной аутентификации не «усложняет» подбор пароля — оно делает его математически бессмысленным, потому что пароля для сравнения просто не существует в протоколе.

Почему SSH — частая цель

SSH — один из самых частых векторов первого доступа на любом хосте с публичным IP: автоматизированные боты сканируют диапазоны адресов и подбирают пароли по словарю в первые минуты после появления нового IP. Если разрешён вход по паролю (`PasswordAuthentication yes`) и пароль недостаточно стойкий, рано или поздно он будет подобран — вопрос только времени и размера словаря атакующего.

В учебном окружении root имеет пароль `toor123`, вход по паролю разрешён, лимит попыток авторизации практически отсутствует (`MaxAuthTries 100`).

Шаг 1. Эксплуатация (роль: hacker)

```
# Заходим под учётную запись-нарушителя
su - hacker

# TARGET_IP уже должен быть определён и явно задан (см. п. 0.5)
echo "$TARGET_IP"

# Подбор пароля root по словарю (учебный словарь лежит в ~/wordlist.txt)
hydra -l root -P ~/wordlist.txt ssh://$TARGET_IP
```

После того как hydra найдёт пароль, подключитесь и прочитайте флаг:

```
# Подключаемся найденным паролем (toor123)
ssh root@$TARGET_IP
# Внутри сессии root читаем флаг
cat /root/flags/flag1.txt
```

> **Скриншот 1.1.** Терминал с меткой `[LAB:...]`: вывод hydra с найденным паролем и содержимое `flag1.txt` после входа по SSH под root.

Шаг 2. Защита (роль: администратор)

Переходим на вход только по ключу — убирает подбор пароля как вектор атаки полностью. Открываем файл конфигурации в текстовом редакторе и вносим изменения вручную, чтобы ясно видеть, что именно меняется:

```
# Возвращаемся в административную учётную запись
exit

sudo nano /etc/ssh/sshd_config
```

В файле найдите (скрипт подготовки стенда уже добавил эти строки с «уязвимыми» значениями) и измените три строки:

Было (уязвимо)	Стало (защищено)	Что это значит
`PermitRootLogin yes`	`PermitRootLogin no`	Запрещает прямой вход по SSH под root вообще, независимо от способа аутентификации. Администратор заходит под обычной учётной записью и повышает права через `sudo` — так в системе остаётся след, под каким именно пользователем действие было выполнено, а самая привилегированная учётная запись не подвержена прямому сетевому перебору.
`PasswordAuthentication yes`	`PasswordAuthentication no`	Отключает проверку пароля в протоколе SSH полностью. Сервер не будет даже предлагать клиенту этот способ аутентификации — остаётся только вход по ключу. Именно эта строка убирает подбор пароля как работающий вектор атаки.

Было (уязвимо)	Стало (защищено)	Что это значит
`MaxAuthTries 100`	`MaxAuthTries 3`	Ограничивает количество попыток аутентификации в рамках одного TCP-соединения, после чего сервер разрывает связь. Это защита второго порядка: даже если `PasswordAuthentication` случайно снова включат (ошибка конфигурации, откат изменений), подбор будет искусственно медленным, потому что после каждых 3 попыток нужно устанавливать новое соединение. **Важно:** `MaxAuthTries` — это не блокировка атакующего, а только ограничение скорости внутри одного соединения; чем он отличается от `maxretry` в fail2ban — разобрано в задании 2.

Сохраните файл (`Ctrl+O`, `Enter`, затем `Ctrl+X` для выхода из `nano`).

Перед перезапуском службы обязательно проверьте синтаксис — опечатка в конфиге иначе проявится только в момент перезапуска и может заблокировать вам доступ:

```
sudo sshd -t
```

Если команда ничего не вывела — синтаксис корректен. Теперь можно безопасно перезапустить службу:

```
sudo systemctl restart sshd
```

Откройте отдельный, новый терминал и проверьте доступ по ключу, прежде чем закрывать текущую сессию — если что-то настроено неверно, у вас останется рабочая сессия, через которую можно откатить изменение.

Общая рекомендация по защите: ключ + `PermitRootLogin no` закрывают подбор пароля полностью; дополнительно порт можно сменить с 22 на нестандартный (директива `Port`) — это не остановит целевую атаку конкретно на вас, но резко снизит фоновый шум от массовых автоматических сканеров, которые проверяют в основном только порт 22.

Шаг 3. Проверка (роль: hacker)

```
su - hacker
echo "$TARGET_IP"
hydra -l root -P ~/wordlist.txt ssh://$TARGET_IP
```

Ожидаемый результат — hydra не находит валидных паролей (все попытки отклонены), поскольку `PasswordAuthentication no` отключает парольную авторизацию на уровне протокола, независимо от того, насколько «удачный» пароль подбирается.

> **Скриншот 1.2.** Терминал с меткой `[LAB:...]`: повторный запуск hydra с результатом «0 valid passwords found» (или аналогичным отказом).

Задание 2 (обязательно). Fail2ban как компенсирующий контроль

Подготовка стенда

Задание 2 выполняется на том же SSH-стенде, что и задание 1, но в сценарии «пароль отключить нельзя»:

```
cd ~/lab_network_protection
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" fail2ban
```

Скрипт снова включает `PasswordAuthentication yes` и `PermitRootLogin yes` (если вы уже отключили их в задании 1) и проверяет, что `root:toor123` работает. Сам fail2ban скрипт

не ставит — его вы устанавливаете вручную в шаге 2, чтобы видеть, что именно он меняет в системе.

Теория

Задания 1 и 4 — это **превентивная** защита: убрать возможность атаки заранее (отключить пароль, закрыть порт). fail2ban устроен принципиально иначе — это **реактивная** защита: он не предотвращает попытку атаки, а распознаёт её по поведению и блокирует источник после того, как эта попытка уже произошла несколько раз.

Задание 1 показало идеальное решение — отключить вход по паролю полностью. Но на практике иногда это невозможно: сторонняя интеграция, legacy-система, сервис, которому по независимым причинам необходим вход по паролю. В таком случае единственная практическая защита — не предотвратить подбор пароля, а сделать его экономически невыгодным.

Почему «host-based» и почему именно «IPS»

- **Host-based** — в отличие от сетевого IDS/IPS (который анализирует трафик на уровне сети, часто на отдельном устройстве), fail2ban работает локально на самом хосте и анализирует **логи этого же хоста** (например, журнал попыток входа по SSH).
- **IPS, а не просто IDS** — IDS (Intrusion *Detection* System) только обнаруживает и сообщает; IPS (Intrusion *Prevention* System) активно вмешивается. fail2ban именно вмешивается: при срабатывании он добавляет правило в firewall (через nftables/iptables), которое реально блокирует трафик с адреса атакующего — это не просто запись в лог «замечена подозрительная активность».

Как это устроено внутри

- **Jail (тюрьма)** — одна настроенная единица защиты, обычно привязанная к одной службе (sshd, веб-сервер и т. д.). У каждого jail свой лог, который он читает, и свои параметры срабатывания.
- **Filter (фильтр)** — регулярное выражение, которое ищет в логе строки о неудачной попытке и извлекает из них IP-адрес (специальный токен <HOST> в регулярном выражении fail2ban). Например, фильтр для sshd ищет строки вида `Failed password for ... from <HOST> port ...`.
- **Action (действие)** — что происходит при срабатывании фильтра нужное число раз. По умолчанию — добавление адреса в специальный firewall-набор (set) на блокировку,

что технически реализуется через вызов nftables/iptables из самого fail2ban.

- **Backend** — каким способом fail2ban отслеживает изменения в логе: `systemd` (читает напрямую из журнала systemd — используется в этой лабораторной, так как sshd на Debian 13 пишет именно туда), `pyinotify` (следит за изменениями файла на диске), `polling` (периодически перечитывает файл — самый медленный и ресурсоёмкий способ, используется как запасной вариант).

Ключевые параметры

Параметр	Что означает
`findtime`	Размер «окна» времени, в течение которого считаются неудачные попытки. Пример: если `findtime = 10m`, то важны только попытки за последние 10 минут — более старые не учитываются.
`maxretry`	Сколько неудачных попыток в пределах `findtime` допускается, прежде чем IP будет заблокирован.
`bantime`	На сколько времени блокируется IP-адрес после превышения `maxretry`.
`bantime.increment` / `bantime.factor`	Если включено, каждая повторная блокировка того же адреса увеличивается (обычно умножается на `factor`) — учит систему «запоминать» настойчивых атакующих и банить их всё дольше.
`ignoreip`	Список адресов, которые fail2ban никогда не блокирует, даже если формально подходят под правило — сюда нужно внести свой собственный рабочий IP, иначе можно заблокировать самого себя во время тестирования.

`MaxAuthTries` в sshd и `maxretry` в fail2ban — это разные вещи

Оба параметра «про попытки», поэтому их постоянно путают. Разница принципиальная:

`MaxAuthTries` — директива **самого sshd**. Она ограничивает число попыток аутентификации **внутри одного уже установленного TCP-соединения**: после превышения sshd просто разрывает это соединение. Он ничего не блокирует и никого не запоминает — атакующий немедленно открывает новое соединение и продолжает с того же IP. Поэтому `MaxAuthTries 3` не останавливает подбор, а лишь замедляет его: вместо непрерывного потока попыток клиент вынужден переподключаться после каждой третьей.

`maxretry` — параметр **fail2ban**, отдельной службы, которая к sshd отношения не имеет. Она читает лог sshd, считает **неудачные попытки с одного IP за окно `findtime` — по всем соединениям сразу** — и, когда их набирается `maxretry`, добавляет IP в firewall-набор на `bantime`. После этого пакеты с этого адреса отбрасываются ещё до того, как их увидит sshd.

Что сравниваем	`MaxAuthTries` (sshd)	`maxretry` (fail2ban)
Где работает	Внутри sshd, на уровне протокола SSH	Вне службы: читает лог, управляет firewall
Что считает	Неудачные попытки в рамках **одного TCP-соединения**	Неудачные попытки с одного IP **за время `findtime`, по всем соединениям**
Что делает при превышении	Разрывает текущее соединение	Блокирует IP на `bantime` на уровне nftables/iptables
Кого затрагивает	Только sshd и только это соединение	Весь хост: все службы и порты для этого адреса
Как обходится	Тривиально — переподключением; скорость падает, подбор продолжается	Только сменой IP (ботнет, прокси, другой канал)

Что сравниваем	MaxAuthTries (sshd)	maxretry (fail2ban)
Когда срабатывает	Сразу и детерминированно, по числу попыток в соединении	С задержкой: после maxretry неудач в окне findtime
Кто настраивает	Администратор в /etc/ssh/sshd_config	Администратор в /etc/fail2ban/jail.local, независимо от sshd
Риск ложных срабатываний	Практически нет	Есть: забыли пароль, скрипт с неверным ключом — и вы забанены

Короткая формулировка для отчёта:

- MaxAuthTries — **ограничитель скорости внутри одного соединения**, не блокировка;
- maxretry — **порог блокировки источника** по совокупности попыток за интервал.

Отсюда два практических вывода. Первый: MaxAuthTries не заменяет fail2ban — hydra просто откроет новое соединение на каждые три попытки. Второй: fail2ban не заменяет MaxAuthTries — он реагирует постфактум, уже после нескольких неудач, и бесполезен против распределённого перебора с множества адресов, каждый из которых не превышает maxretry.

Ограничения метода

fail2ban не защищает от атаки, распределённой по множеству разных IP-адресов (каждый отдельный адрес может не превысить maxretry), и не защищает от того, что вообще не оставляет характерного следа в логе. Это осознанный компромисс: он эффективен именно против массового автоматического перебора с одного источника — самого частого на практике случая — но не является заменой устранения самой причины уязвимости (как в задании 1).

Шаг 1. Эксплуатация (сценарий + атака)

Подготовка уже вернула вход по паролю и вход root (PasswordAuthentication yes, PermitRootLogin yes) — это и есть сценарий «пароль отключить нельзя». Убедитесь, что служба перезапущена:

```
sudo sshd -T | grep -i passwordauthentication
sudo systemctl restart sshd
```

Теперь от hacker — подбор пароля, как в задании 1:

```
su - hacker
echo "$TARGET_IP"
hydra -l root -P ~/wordlist.txt ssh://$TARGET_IP
```

Без fail2ban hydra может перебирать пароли сколь угодно долго без каких-либо последствий для атакующего IP: MaxAuthTries в конфиге не мешает — после исчерпания попыток в соединении hydra просто подключается заново.

> **Скриншот 2.1.** Терминал с меткой [LAB:...]: hydra выполняет множество попыток подряд без блокировки.

Шаг 2. Защита (роль: администратор)

```
exit
sudo apt-get install -y fail2ban
sudo nano /etc/fail2ban/jail.local
```

Впишите содержимое (новый файл — редактируется jail.local, а не jail.conf, потому что jail.conf перезатирается при обновлении пакета):

```
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 4
bantime.increment = true
bantime.factor = 2
ignoreip = 127.0.0.1/8 ::1

[sshd]
enabled = true
port    = 22
logpath = %(sshd_log)s
backend = systemd
```

Где: - bantime = 1h — первая блокировка держится час; - findtime = 10m — учитываются только попытки за последние 10 минут; - maxretry = 4 — после 4 неудачных попыток в этом окне IP блокируется (это счётчик fail2ban по всем соединениям, а не MaxAuthTries внутри sshd — см. таблицу в теории); - bantime.increment = true и bantime.factor = 2 — каждая следующая блокировка того же адреса длится вдвое дольше предыдущей (1ч → 2ч → 4ч → ...); - ignoreip — ваш собственный адрес сюда не входит по умолчанию; если тестируете с того же IP, с которого администрируете систему, добавьте его в этот

список через пробел, иначе рискуете заблокировать сами себя; `[sshd]` — секция-jail для службы sshd: `enabled = true` включает именно этот jail (в комплекте fail2ban их десятки, под разные службы, большинство по умолчанию отключены); `backend = systemd` указывает читать журнал через systemd, а не искать лог-файл на диске.

Примените конфигурацию:

```
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

> **Примечание.** На учебном стенде fail2ban срабатывает и реально блокирует адрес, но может не писать в журнал и не показывать забаненный адрес в `fail2ban-client status sshd` — это нормально, так было на занятии у всех. Не ориентируйтесь на «красивый» статус: факт блокировки подтверждается **поведением атаки** — повторный запуск hydra обрывается (таймауты вместо ответов sshd, `Connection refused`), и подбор дальше не идёт.

Общая рекомендация по защите: fail2ban — компенсирующий, а не основной контроль; он снижает скорость подбора пароля на порядки, но не исключает его математически, в отличие от отключения парольного входа (задание 1). Использовать его как единственную защиту обоснованно только тогда, когда отключить пароль действительно нельзя.

Шаг 3. Проверка (роль: hacker)

```
su - hacker
echo "$TARGET_IP"
hydra -l root -P ~/wordlist.txt ssh://$TARGET_IP
```

Ожидаемый результат — после нескольких неудачных попыток (по умолчанию 4 за 10 минут) IP-адрес блокируется, дальнейшие попытки подключения от этого адреса отклоняются на уровне сети, а не SSH (hydra начинает получать таймауты вместо ответа sshd). Поэтому доказательство работы fail2ban — **повторная атака, которая захлёбывается**, а не вывод `fail2ban-client status`: статус на этом стенде бан часто не отображает.

```
exit
sudo fail2ban-client status sshd
```

> **Скриншот 2.2.** Терминал с меткой `[LAB:...]` на машине атакующего: **неудачная атака после блокировки** — повторный запуск hydra по тому

же адресу, где попытки обрываются таймаутами или `Connection refused`, и подбор не продолжается. Снимок `fail2ban-client status sshd` не требуется: он может не показывать бан (см. примечание в шаге 2).

После завершения задания верните защищённые значения из задания 1 — `PasswordAuthentication no` и `PermitRootLogin no` (если выполняли задание 1): включённый вход по паролю был временным сценарием только для демонстрации fail2ban.

Теория: Docker, образы, контейнеры и изоляция

Образ vs контейнер

- **Образ (image)** — неизменяемый шаблон: файловая система приложения (бинарники, библиотеки, конфиги) плюс метаданные (команда запуска, переменные окружения, открытые порты). Образ сам по себе ничего не выполняет — это файл на диске (точнее, набор слоёв, см. ниже).
- **Контейнер (container)** — запущенный экземпляр образа: тот же набор файлов плюс один (или несколько) реально работающих процессов, собственное сетевое пространство, собственный writable-слой сверху образа. Один образ может породить множество независимых контейнеров.

Аналогия: образ — это класс в программировании, контейнер — это объект, созданный из этого класса.

Из чего состоит образ: слои и UnionFS

Образ Docker собирается из слоёв (layers) — каждая инструкция в Dockerfile (`RUN`, `COPY` и т. п.) создаёт новый слой, содержащий только *изменения* относительно предыдущего. Слои накладываются друг на друга через объединяющую файловую систему (обычно **OverlayFS** на Linux): снизу — read-only слои образа, сверху — один writable-слой, уникальный для каждого контейнера. Когда процесс в контейнере изменяет файл из нижнего слоя, срабатывает **copy-on-write**: файл копируется в writable-слой и меняется уже там, нижние слои не трогаются.

Практическое следствие: несколько контейнеров из одного образа безопасно делят одни и те же read-only слои на диске (экономия места), и удаление контейнера просто отбрасывает его writable-слой — образ не затрагивается.

Как работает изоляция: это не «лёгкая виртуальная машина»

Ключевая вещь, которую нужно понимать: **контейнер — это обычный процесс Linux**, работающий на том же ядре, что и хост. Никакой отдельной гостевой ОС нет (в этом и отличие от VirtualBox/VMware — там гостевая ОС и своё ядро). Изоляция контейнера строится из нескольких независимых механизмов ядра Linux, которые docker просто включает одновременно:

Механизм	Что он изолирует
Namespaces (пространства имён)	Что процесс *видит*
cgroups (control groups)	Сколько ресурсов процесс может *использовать*
Capabilities	Какие привилегированные операции процесс может *выполнять*
Seccomp	Какие системные вызовы процессу разрешены вообще

Namespaces — что процесс видит

Каждый вид namespace «показывает» процессу его собственную, урезанную версию соответствующего ресурса ядра:

- **PID namespace** — процесс внутри контейнера видит только процессы своего namespace; его собственный `PID 1` — это не настоящий `PID 1` хоста. `ps aux` внутри контейнера не покажет процессы хоста.
- **Network namespace** — отдельный набор сетевых интерфейсов, таблица маршрутизации, порты. Поэтому один и тот же порт 8080 может одновременно использоваться и на хосте, и в десяти разных контейнерах — они в разных network namespace.
- **Mount namespace** — собственное дерево точек монтирования: контейнер видит свою файловую систему (слои образа + writable-слой), а не файловую систему хоста целиком — *если только её явно не примонтировать внутрь* (как и происходит в задании 3).
- **UTS namespace** — собственное имя хоста (`hostname`).
- **IPC namespace** — отдельные механизмы межпроцессного взаимодействия (System V IPC, POSIX-очереди).
- **User namespace** (опционален, часто не включён по умолчанию) — отображение `uid`/`gid`: `root` внутри контейнера

может соответствовать непривилегированному пользователю на хосте. Без user namespace root в контейнере — это буквально uid=0, тот же root, что и на хосте.

cgroups — сколько ресурсов разрешено

cgroups ограничивают CPU, память, I/O, количество процессов для данной группы процессов — именно через cgroups работают --memory, --cpus, --pids-limit в docker run. Без лимитов один контейнер может исчерпать память или CPU всего хоста.

Capabilities — что разрешено делать, даже если ты root

Классический Linux-root может всё. Capabilities разбивают «всё» на отдельные флаги (около 40): CAP_NET_ADMIN (менять сетевые настройки), CAP_SYS_ADMIN (широкий набор административных операций), CAP_SYS_MODULE (загружать модули ядра) и т. д. По умолчанию Docker запускает контейнер с урезанным набором capabilities, а не с полным набором root — но при --privileged Docker выдаёт **все** capabilities и отключает другие ограничения (см. ниже).

Seccomp — какие системные вызовы разрешены

Seccomp-профиль — это белый список системных вызовов (syscalls), которые процессу разрешено выполнять. Docker применяет вполне строгий профиль по умолчанию, блокирующий десятки редко нужных и потенциально опасных syscalls (например, напрямую загружать модули ядра, манипулировать пространствами имён в обход контейнера). --privileged снова отключает это ограничение.

Почему --privileged + -v /:/hostfs — это фактически отказ от изоляции

Флаг --privileged: - выдаёт контейнеру **все** capabilities (а не урезанный набор); - отключает seccomp-профиль; - снимает ограничения на доступ к устройствам (/dev/*).

Монтирование -v /:/hostfs: - прокидывает **весь корень файловой системы хоста** внутрь контейнера как обычную директорию /hostfs.

По отдельности каждая из этих настроек уже рискованна; вместе они означают, что процесс внутри контейнера, с правами root (без user namespace — а это стандартная настройка), имеющий все capabilities и прямой доступ к файловой системе хоста, **практически эквивалентен root-процессу на самом**

хосте — namespaces по-прежнему изолируют *видимость* процессов и сети, но этого недостаточно, когда у контейнера прямой путь к файлам хоста и возможность изменить их права/владельца.

Это ровно та конфигурация, которую задание 3 просит вас сначала проэксплуатировать, а затем устранить.

Задание 3 (на выбор). Выход за пределы Docker-контейнера

Подготовка стенда

```
cd ~/lab_network_protection
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" docker
```

Скрипт ставит `docker.io`, копирует приложение в `/opt/vulnerable_app`, собирает локальный образ `lab-vuln-app` (на основе `python:3-slim` + пакет `iputils-ping` — в базовом slim-образе утилиты `ping` нет) и запускает контейнер `vuln_app` с `--privileged` и `-v /:/hostfs`, а также создаёт `flag3`. Если вы выбрали задание 4 вместо этого — Docker вам не понадобится и установлен не будет.

> **Если контейнер уже был поднят старой версией скрипта** (без `ping`) и шаг 1 возвращает `ping: not found`, пересоберите стенд или доустановите утилиту в работающий контейнер:
>
> ```
> sudo ./scripts/lab_network_protection_setup.sh "<ваш идентификатор>" docker
> # либо, без пересоздания контейнера:
> docker exec vuln_app sh -c "apt-get update && apt-get install -y iputils-pin
> ```

Теория задания

Это задание напрямую использует раздел «Теория: Docker…» выше — короткое резюме применительно к сценарию: веб-приложение внутри контейнера содержит уязвимость command injection (классическая ошибка — непроверенный пользовательский ввод подставляется в системную команду), а сам контейнер запущен с `--privileged` и с примонтированным внутрь корнем хоста (`-v /:/hostfs`). Это означает, что *любая* возможность выполнить команду внутри контейнера — это, по сути, возможность выполнить команду на хосте: изоляция namespaces никуда не пропадает, но путь к файлам хоста

открыт напрямую, а привилегий достаточно, чтобы этим путём воспользоваться без ограничений.

Шаг 1. Эксплуатация (роль: hacker)

```
su - hacker
echo "$TARGET_IP"

# Обычный, "легитимный" запрос к приложению
curl "http://$TARGET_IP:8080/ping?host=127.0.0.1"

# Эксплойт: command injection в параметре host — через ";" подставляется
# вторая команда, которая читает флаг с ХОСТА через смонтированный /hostfs
curl "http://$TARGET_IP:8080/ping?host=127.0.0.1;cat+/hostfs/root/flags/flag3.tx
```

Ответ приложения будет содержать вывод `ping`, а следом — содержимое `flag3.txt`, прочитанного напрямую с файловой системы хоста из процесса, запущенного внутри контейнера.

> **Скриншот 3.1.** Терминал с меткой `[LAB:...]`: запрос с command injection и содержимое `flag3.txt` в ответе.

Шаг 2. Защита (роль: администратор)

Исправление — на уровне конфигурации контейнера: убрать привилегии и доступ к хосту, которые не нужны приложению для работы. Саму уязвимость command injection в коде приложения в этом задании не трогаем — цель именно в том, чтобы показать, что правильная конфигурация окружения ограничивает ущерб даже от уже существующей уязвимости в коде.

```
exit
docker rm -f vuln_app
```

Пересоздаём контейнер с другим набором флагов. Ниже — таблица с объяснением каждого из них, прежде чем выполнять команду:

Флаг	Значение	Что он ограничивает и почему
`--user nobody`	Процесс внутри контейнера работает как пользователь nobody, а не как root	Даже если изоляция где-то окажется недостаточной, у процесса нет прав root ни внутри контейнера, ни тем более снаружи
`--read-only`	Корневая файловая система контейнера (образ + writable-слой) становится недоступна на запись	Если атакующий всё же получит выполнение команд, он не сможет ничего записать на диск контейнера — не получится, например, положить туда свой исполняемый файл
`--tmpfs /tmp`	Отдельная временная файловая система в памяти, смонтированная поверх `--read-only`, доступна на запись только в `/tmp`	Некоторым приложениям всё же нужно куда-то писать временные файлы — даём им изолированное место, которое не сохраняется между перезапусками контейнера

Флаг	Значение	Что он ограничивает и почему
`--cap-drop ALL`	Снимает у контейнера абсолютно все Linux capabilities	Без этого флага контейнер по умолчанию получает небольшой, но не нулевой набор capabilities (например, `CAP_NET_BIND_SERVICE`); явно отбираем всё, а затем добавляем точечно только то, что реально нужно (здесь — ничего не требуется, приложению достаточно открыть TCP-сокет на непривилегированном порту)
`--security-opt no-new-privileges`	Запрещает процессу и его потомкам повышать привилегии через SUID-биты, файловые capabilities и аналогичные механизмы	Та же идея, что и снятие SUID-бита в лабораторной по эскалации привилегий, только здесь ограничение накладывается на весь контейнер целиком, а не на один конкретный файл
`--memory=256m`	Жёсткий лимит памяти для контейнера (через cgroups)	Не позволяет одному скомпрометированному или просто ошибочно работающему контейнеру исчерпать память всего хоста

Флаг	Значение	Что он ограничивает и почему
`--pids-limit=64`	Максимальное количество процессов внутри контейнера (через cgroups)	Защита от fork-бомб и аналогичных атак на исчерпание ресурсов
`-v /opt/vulnerable_app:/app:ro`	Код приложения монтируется внутрь контейнера **только на чтение** (`:ro`)	Контейнер может читать и выполнять код, но не может его изменить — и, что важнее, здесь больше нет `-v /:/hostfs`: корень хоста вообще не виден изнутри контейнера

Сама команда запуска:

```
docker run -d \
    --name vuln_app \
    --user nobody \
    --read-only \
    --tmpfs /tmp \
    --cap-drop ALL \
    --security-opt no-new-privileges \
    --memory=256m \
    --pids-limit=64 \
    -p 8080:8080 \
    -v /opt/vulnerable_app:/app:ro \
    -w /app \
    lab-vuln-app \
    python3 vulnerable_app.py
```

Дополнительно, за пределами самого контейнера — применить noexec к /tmp на хосте: даже если что-то всё же окажется записано во временный каталог хоста (не контейнера — это отдельная защита уровня хоста), его нельзя будет напрямую запустить.

```
sudo nano /etc/fstab
```

Добавьте в конец файла строку:

```
tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev,size=512M 0 0
```

где: - noexec — ничего нельзя выполнить непосредственно из /tmp (большинство простых пост-эксплуатационных payload'ов сначала оказываются именно там); - nosuid — SUID-биты на файлах, помещённых сюда, игнорируются; - nodev — нельзя создавать файлы устройств.

Примените изменение без перезагрузки:

```
sudo mount -o remount /tmp
```

Общая рекомендация по защите: уязвимость в коде приложения (command injection) устраняется на уровне кода и является отдельной задачей разработки; но на уровне инфраструктуры задача администратора — не давать контейнеру больше возможностей, чем ему реально требуется, именно для того, чтобы баг в приложении не превращался в компрометацию хоста. Это и есть принцип defense in depth: два независимых слоя защиты (код + конфигурация окружения), каждый из которых может подвести сам по себе.

Шаг 3. Проверка (роль: hacker)

```
su - hacker
echo "$TARGET_IP"
curl "http://$TARGET_IP:8080/ping?host=127.0.0.1;cat+/hostfs/root/flags/flag3.tx
curl "http://$TARGET_IP:8080/ping?host=127.0.0.1;id"
```

Ожидаемый результат — command injection в самом приложении всё ещё технически происходит (ping ... ; id всё ещё выполняется, потому что баг в коде не исправлялся), но: - /hostfs/... вообще не существует внутри контейнера — хост больше не примонтирован, команда вернёт ошибку «No such file or directory»; - id показывает непривилегированного пользователя (nobody), а не root; - попытка что-либо записать получит Read-only file system, кроме /tmp.

> **Скриншот 3.2.** Терминал с меткой [LAB:...]: тот же запрос, но без флага в ответе, плюс вывод curl ...;id, показывающий непривилегированного пользователя.

Теория: nftables и фильтрация пакетов

Задание 4 использует nftables как основной инструмент защиты — разберём, как он устроен, до того, как переходить к самому заданию.

Netfilter и nftables

Netfilter — подсистема ядра Linux, которая перехватывает каждый проходящий через сетевой стек пакет в определённых точках и позволяет принять решение: пропустить, отбросить, изменить. **nftables** — современный интерфейс для управления этой подсистемой (замена устаревшего `iptables`; команда `iptables` на Debian 13 всё ещё работает, но как транслятор поверх nftables). Решение о допуске пакета принимается **до** того, как он попадёт в приложение — то есть это защита на уровень ниже, чем всё, что делает сама служба (sshd, веб-сервер и т. д.).

Таблицы, цепочки, хуки, политика

- **Таблица (table)** — верхний уровень организации правил, обычно разделяется по семейству адресов (`ip`, `ip6`, `inet` — сразу оба).
- **Цепочка (chain)** — упорядоченный список правил. У базовой цепочки есть **hook** — точка в пути пакета, где она подключается: `input` (пакеты, предназначенные этому хосту), `output` (пакеты, которые этот хост отправляет), `forward` (пакеты, проходящие транзитом, если хост — маршрутизатор).
- **Priority** — порядок, в котором выполняются цепочки с одним и тем же hook, если их несколько; `0` — нейтральное значение по умолчанию.
- **Policy (политика)** — что делать с пакетом, если ни одно правило в цепочке явно не вынесло вердикт. `policy drop` — пакет отбрасывается по умолчанию, пропускается только то, что разрешено явным правилом (принцип **allowlist** — белый список: «запрещено всё, что не разрешено явно»). Это принципиально безопаснее, чем `policy accept` с отдельными правилами `drop` для известных угроз (**denylist** — чёрный список), потому что denylist защищает только от того, что вы уже предвидели, а allowlist по умолчанию защищает и от того, что вы не предвидели.

Отслеживание соединений (conntrack) и состояния

nftables — **стейтфул** (stateful) фильтр: он запоминает установленные соединения через модуль `conntrack` и классифицирует каждый пакет по состоянию:

- **`new`** — первый пакет нового соединения (например, SYN для TCP).

- **established** — пакет относится к уже установленному, ранее разрешённому соединению.
- **related** — пакет логически связан с существующим соединением, но не является его частью напрямую (классический пример — ICMP-ошибка в ответ на TCP-соединение).
- **invalid** — пакет не соответствует ни одному известному соединению и не является корректным началом нового (признак подделки, сканирования нестандартными флагами, и т. п.).

Практический смысл: вы пишете правило `ct state established,related accept` **один раз**, и после этого не нужно отдельно разрешать входящие пакеты-ответы на ваши собственные исходящие соединения (например, ответ DNS-сервера на ваш запрос) — conntrack уже знает, что это часть разрешённого обмена. Без этого механизма пришлось бы писать по отдельному правилу на каждое направление каждого протокола, что практически нереализуемо.

Ограничение частоты (rate limiting)

Конструкция `limit rate N/second` (или `/minute`) реализует алгоритм, похожий на «ведро с токенами» (token bucket): пакеты пропускаются, пока не превышена заданная средняя скорость, после чего избыточные отбрасываются (или не подпадают под правило — в зависимости от того, что стоит после `limit`). Это не блокирует атакующего навсегда, но делает перебор на порядки медленнее — действие того же рода, что и `MaxAuthTries` в задании 1 и `maxretry` в задании 2, только на уровне пакетов, а не попыток аутентификации или логов.

Как это соберётся в задании 4

Правило будет выглядеть как сочетание четырёх идей выше: `policy drop` по умолчанию (allowlist), `iif lo accept` (трафику на loopback всегда доверяем — через него общаются локальные процессы друг с другом, блокировать его обычно означает ломать саму систему), `ct state established,related accept` (разрешить ответы на наш собственный трафик без лишних правил), и явные правила с `limit rate` только для портов, которые должны быть доступны снаружи. Любой порт, не упомянутый явно, окажется закрыт политикой `drop` — в том числе порт 9000 из задания 4.

Задание 4 (на выбор). Избыточно открытый сетевой сервис

Подготовка стенда

```
cd ~/lab_network_protection
sudo ./scripts/lab_network_protection_setup.sh "Ivanov_I_I" firewall
```

Скрипт поднимает «забытую» диагностическую службу `lab-debug.service` на `0.0.0.0:9000` и создаёт `flag2`, а также ставит `nmap` и `nc`. **Docker при этом не устанавливается** — для этого задания он не нужен.

Теория задания

Любой процесс, слушающий на `0.0.0.0` (а не на `127.0.0.1`), доступен с любого адреса, с которого в принципе есть сетевой маршрут до хоста — независимо от того, предполагался ли такой доступ изначально:

- **`0.0.0.0`** — означает «слушать на всех сетевых интерфейсах, которые есть у этой машины» (внешний, loopback, VPN и т. д.).
- **`127.0.0.1`** — означает «слушать только на loopback-интерфейсе», то есть принимать соединения только от процессов, работающих на этой же машине. Снаружи по сети до такого адреса просто не существует маршрута.

Забытые debug-интерфейсы, админ-панели «только для внутреннего использования», тестовые сервисы — классический источник утечки информации, даже без дополнительной эксплуатации: сервис сам раскрывает то, что знает, любому, кто подключится, если привязан к `0.0.0.0` по ошибке (или по умолчанию — многие фреймворки разработки по умолчанию слушают именно так, чтобы было удобнее тестировать).

В окружении на порту 9000 слушает «забытая» диагностическая служба `lab-debug.service` (файл `/opt/lab-net/debug_status_service.py`), которая на любое TCP-подключение отдаёт служебную информацию, включая `flag2`, без какой-либо авторизации.

Шаг 1. Эксплуатация (роль: hacker)

```
su - hacker
echo "$TARGET_IP"

# Разведка: какие порты открыты на целевом хосте
nmap -sV $TARGET_IP
```

```
# На порту 9000 обнаружен неизвестный сервис — подключаемся и смотрим, что он от
nc $TARGET_IP 9000
```

Служба ответит баннером, содержащим `flag2`, без какой-либо авторизации.

> **Скриншот 4.1.** Терминал с меткой `[LAB:...]`: вывод `nmap`, показывающий открытый порт 9000, и вывод `nc`, содержащий `flag2`.

Шаг 2. Защита (роль: администратор)

Правильная защита — на двух независимых уровнях: сначала убрать сервис из публичного доступа на уровне его собственной конфигурации, затем — firewall как второй рубеж (это и есть defense in depth — несколько независимых слоёв защиты, а не один «починили и забыли»).

Уровень 1: привязка службы только к loopback

```
exit
sudo nano /opt/lab-net/debug_status_service.py
```

Найдите строку:

```
HOST = "0.0.0.0"
```

и замените на:

```
HOST = "127.0.0.1"
```

Сохраните файл и перезапустите службу, чтобы изменение применилось:

```
sudo systemctl restart lab-debug.service
```

Теперь служба физически не принимает соединения ни с одного внешнего адреса — она даже не пытается слушать на внешнем интерфейсе.

Уровень 2: nftables с политикой default-drop

Даже если какая-то другая служба по ошибке останется привязана к `0.0.0.0`, firewall должен быть вторым, самостоятельным рубежом. Создайте файл конфигурации:

```
sudo nano /etc/nftables.conf
```

Впишите в него следующее содержимое (объяснение каждой строки — в теоретическом разделе выше и в комментариях ниже):

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;

     # Loopback — локальные процессы общаются друг с другом через
     # 127.0.0.1/::1, блокировать это нельзя, иначе ломается сама система
        iif lo accept

     # Пакеты, относящиеся к уже установленным (или логически связанным)
     # соединениям, которые мы сами инициировали или уже разрешили —

       # разрешаем без отдельного правила на каждый случай
        ct state established,related accept

     # Пакеты, не соответствующие ни одному известному соединению и не
             #  являющиеся  корректным  началом  нового  —
явный признак подделки
      # или нестандартного сканирования, отбрасываем сразу
        ct state invalid drop

     # ICMP echo-request (ping) — штатная диагностика, но с ограничением
     # частоты, чтобы не использовали как инструмент нагрузки
     ip protocol icmp icmp type echo-request limit rate 5/second accept

     # SSH (порт 22): разрешаем только НОВЫЕ соединения и ограничиваем
     # частоту их установления — не ограничивает обмен данными внутри
     # уже установленной сессии, только скорость появления новых попыток
      tcp dport 22 ct state new limit rate 10/minute accept

     # Порт 9000 здесь НЕ упомянут. Любой пакет на любой порт, который
     # не был явно разрешён выше, попадает под политику цепочки (policy
     # drop) и отбрасывается — это и есть принцип allowlist из теории.
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
    }

    chain output {
       type filter hook output priority 0; policy accept;
    }
}
```

Сохраните файл и примените его:

```
# Включаем автозапуск правил при загрузке системы
sudo systemctl enable nftables

# Загружаем правила из файла немедленно, не дожидаясь перезагрузки
sudo nft -f /etc/nftables.conf

# Выводим то, что реально загружено в ядро — проверяем, что совпадает
# с тем, что мы написали в файле (опечатка в комментарии nftables
# просто проигнорирует, а опечатка в правиле может привести к
# неожиданному результату)
sudo nft list ruleset
```

Общая рекомендация по защите: связывать внутренние/диагностические сервисы с 127.0.0.1 по умолчанию — это должно быть осознанным решением открыть их наружу, а не случайным следствием того, что адрес привязки не был указан явно. Firewall с политикой drop по умолчанию — это второй, независимый рубеж на случай, если первый всё-таки где-то пропущен (а он будет пропущен — в большой системе с десятками служб невозможно гарантировать, что каждая из них правильно сконфигурирована).

Шаг 3. Проверка (роль: hacker)

```
su - hacker
echo "$TARGET_IP"
nmap -sV $TARGET_IP
nc $TARGET_IP 9000
```

Ожидаемый результат — порт 9000 либо не отображается в выводе nmap вовсе (блокировка на firewall видна как «filtered» или порт просто отсутствует в списке открытых), либо соединение через nc не устанавливается (Connection refused/таймаут): сервис больше не достижим снаружи ни на уровне своей конфигурации (слушает только на loopback), ни на уровне firewall (порт не в списке разрешённых).

> **Скриншот 4.2.** Терминал с меткой [LAB:...]: nmap без порта 9000 в списке и/или nc с ошибкой подключения.

Итоговая форма отчёта

> **Обязательны задания 1 и 2. Из заданий 3 и 4 достаточно выполнить одно — на выбор.**

Для каждого выполненного задания в отчёт включаются:

1. Номер и название задания.
2. Скриншот эксплуатации (с флагом/результатом и меткой `[LAB:...]`).
3. Команды/изменения, применённые для защиты.
4. Скриншот проверки (с отказом в доступе и меткой `[LAB:...]`).
5. Один-два предложения: в чём первопричина уязвимости и что именно в конфигурации изменилось, чтобы её устранить.

№	Задание	Статус	Флаг/результат получен (экспл.)	Атака не работает (защита)
1	SSH: подбор пароля	обязательное	flag1	[ ]
2	Fail2ban: блокировка перебора	обязательное	бан IP в `fail2ban-client status sshd`	[ ]
3	Выход за пределы Docker-контейнера	на выбор	flag3	[ ]
4	Избыточно открытый сервис (nftables)	на выбор	flag2	[ ]

Отчёты без видимой на всех скриншотах метки `LAB_TOKEN`, либо с меткой, не совпадающей с той, что выдал собственный запуск `lab_network_protection_setup.sh` студента, к проверке не принимаются.

Состав пакета файлов

```
app/vulnerable_app.py         — уязвимое приложение (command injection), задание 3
app/debug_status_service.py           — "забытая" debug-
служба на порту 9000, задание 4
scripts/lab_network_protection_setup.sh — разворачивает стенд под выбранное задани
README.md                  — этот документ (в репозитории: `занятие 3 …/лабораторная
```